

# Graphics and Music Challenges for Classic-Style Computer Applications

Peter Occil

This version of the document is dated 2026-07-26.

**Peter Occil**

The following are challenges I make to the computer community, relating to:

- **Graphics for classic-style game development.**
- **MIDI music synthesis for classic-style games and apps.**
- **Tileable wallpapers with limited colors and resolution<sup>1</sup>.**
- **Other challenges and projects.**

All may interest 1990s computer users.

## 1 Contents

- **Contents**
- **Graphics Challenge for Classic-Style Games**
  - **The Specification**
  - **Classic Graphics in Scope**
  - **Optional Limits**
  - **Notes on Specification**
    - \* **Screen resolutions**
    - \* **Frame rate**
    - \* **3-D graphics**
    - \* **Arcade game graphics**
    - \* **Screen image effects (filters)**
    - \* **Sounds**
    - \* **Memory**
  - **Seeking Comments**
  - **Further Reading**
- **Building a Public-Domain music synthesis library and instrument banks**
- **Other Challenges and Projects**
  - **Classic desktop wallpaper**
  - **Button and border styles for classic interfaces**
  - **Sound bank development guide**
  - **Guide for creating 3-D models in the pre-2000 style**
  - **Book-form tutorial on pre-2000 computer graphics programming**
- **Acknowledgments**
- **License**
- **Notes**

---

<sup>1</sup><https://peteroupc.github.io/classic-wallpaper>

## 2 Graphics Challenge for Classic-Style Games

An interesting challenge for game developers, relating to designing games with classic graphics that run on an exceptional variety of modern and recent computers.

In this document, *classic graphics* generally means two- or three-dimensional graphics achieved by video games from 1999 or earlier, before the advent of programmable “shaders”. For details, see “**Classic Graphics in Scope**”, later. (To summarize: In general, a game screen of  $640 \times 480$  or smaller, up to 12,800 3-D polygons at a time [fewer if the game screen is smaller], and tile- or sprite-based 2-D visuals are involved.)

This challenge is intended to encourage the making of—

- modern video games that simulate pre-2000 graphics and run with very low resource requirements (say, 64 million bytes of memory or less) and even on very low-end computers (say, those that date from 2010 or earlier or support Windows 7, Windows XP, or even older), and
- graphics engines (especially free and open-source ones) devoted to pre-2000 visuals and meant for developing such modern video games.<sup>2</sup>

Most desktop and laptop computers from 2010 on, and most smartphones from 2016 on, can draw even high-quality pre-2000 graphics using only software — without relying on specialized video cards — at  $640 \times 480$  pixels or smaller, screen resolutions typically targeted by video games in the 1990s and earlier.<sup>3</sup>

The challenge sets an *upper bound* on the kind of computer graphics that are of interest. Further **constraints to graphics computation** (such as memory, resource, color, resolution, or triangle limits) are highly encouraged. It is also encouraged to establish a lean programming interface for this specification; for details, see “**Lean Programming Interfaces for Classic Graphics**”<sup>4</sup>.

### 2.1 The Specification

Limit 3-D graphics to the following:<sup>5</sup>

1. The maximum number of primitives that can be shown at a time is equal to screen width times screen height divided by 24.<sup>6</sup>
  - A *primitive* is either a triangle or a line segment. An application may also consider a convex quadrilateral to be a primitive.
  - Each vertex of the primitive points to a vertex from the vertex list described later.
  - Each primitive can be translucent.
2. The maximum number of vertices that can be displayed at a time is 3 times the maximum number of primitives.

---

<sup>2</sup>The following are examples of a graphics library that follows the spirit, even if not the letter, of this specification: *Tilengine*, *kit*, *DOS-like*, *raylib’s rlsw software renderer*. Michal Strehovský published an **interesting technique to create small game applications**, and so did Jani Peltonen. I offer a **template for Win32 applications** that supports 8- and 24-bit-per-pixel game images and paints finished game images using *StretchDIBits*. By contrast, even though today’s Unity and Unreal game engines can allow the making of games with “classic graphics”, they are far from lightweight and their use is not within the spirit of this challenge. <https://github.com/megamarc/Tilengine> <https://github.com/rxi/kit/> <https://github.com/mattiasgustavsson/dos-like> <https://github.com/raysan5/raylib> <https://migeel.sk/blog/2024/01/02/building-a-self-contained-game-in-csharp-under-2-kilobytes/> <https://www.codeslow.com/2019/12/tiny-windows-executable-in-rust.html> <https://gist.github.com/peteroupc/fla5d8e45e27123b86b284271cfd802b>

<sup>3</sup>A computer has adequate performance for pre-2000 graphics if it achieves a score of—(a) 3108 or more 3D marks on the 3DMark2000 benchmark ( $640 \times 480$ ) when run without hardware acceleration, or (b) 195 or greater on the 3DMark2000 CPU speed test. Both figures correspond to the running of two visually demanding 3-D demos, at three levels of detail each, at 60 frames per second (adjusted downward as needed if a demo’s detail level averages more than 12,800 triangles per frame; see the section “Test Descriptions” in the 3DMark2000 help).

<sup>4</sup><https://peteroupc.github.io/graphicsapi.html>

<sup>5</sup>One editor specialized for creating classic 3-D models is the open-source tool *Blockbench*. For classic 3-D scenes, there is the open-source tool *Trenchbroom*. <https://www.blockbench.net/> <https://trenchbroom.github.io/>

<sup>6</sup>This is also known as *visible primitives* or *visible primitives per frame*; in the case of polygons or triangles, this is also called *visible polygons (per frame)* or *visible triangles (per frame)*.

- A *vertex* consists of an XYZ position, an XY texture coordinate, and a red–green–blue vertex color.
  - Each vertex color follows this color format: The red, green, and blue components occupy up to 5 bits each.<sup>7</sup>
3. Each *texture* (an image that is applied to the surface of 3-D objects)—
    - is in a 16-bit-per-pixel format, where each pixel has the vertex color format given earlier, or
    - is in a 1-, 2-, 4-, or 8-bit-per-pixel format and has a table of colors with that color format.
  4. The width and height of each texture are each powers of 2.
  5. A texture’s maximum width, in pixels, is the larger of the screen width and the screen height, but not more than 256. A texture’s maximum height is the same as the maximum texture width.
  6. Textures may contain transparent pixels.
  7. Image files used by the game should not store “pre-pixelated” textures. A “pre-pixelated” image results when an image is enlarged in advance with point filtering (also called nearest-neighbor filtering), with the result that some or all of the resulting image’s rows and columns are repeated.
  8. For 3-D graphics, Z buffering (depth buffering), flat shading, Gouraud shading, per-vertex specular highlighting, per-vertex depth-based fog, line drawing (by approximating the line at integer coordinates), two-texture blending, MIP mapping, source alpha blending, and destination alpha blending are supported.<sup>8</sup> Bilinear filtering and edge antialiasing (smoothing)<sup>9</sup> are optional.
  9. 3-D primitives should undergo perspective correction, but this is optional.<sup>10</sup>

**Example:** For a “screen resolution” (see later) of  $640 \times 480$  pixels, no more than 12,800 primitives ( $640 \times 480 / 24$ ) and 38,400 vertices can be shown at a time, and the maximum texture size is  $256 \times 256$  pixels. For  $320 \times 240$  pixels, the maximums are 3200 primitives, 9600 vertices, and textures of  $256 \times 256$  pixels. For  $320 \times 200$  pixels, the maximums are 2666 primitives, 8000 vertices, and textures of  $256 \times 256$  pixels.

Limit 2-D graphics to the following:<sup>11</sup>

1. Layers:
  1. Up to four *2-D layers* can be displayed at a time. Each 2-D layer is a rectangular array of references to *tiles* (see later), and can also be called a *tile map*.<sup>12</sup>
  2. Up to two of the 2-D layers can undergo a 2-D affine transformation.
  3. If 3-D graphics are being displayed, one of the 2-D layers is replaced with a *3-D layer*, which is an image on which the 3-D graphics are drawn. The 2-D layer replaced this way can vary over time.
  4. The 2-D layers may contain transparent pixels. The 3-D layer may contain transparent and translucent (semitransparent) pixels.<sup>13</sup>

<sup>7</sup>If there is interest, this format may instead be: The red and blue components occupy 5 bits each; the green component, 6 bits.

<sup>8</sup>*Quake* (1996) also employed *subdivision rasterization* for drawing small and relatively distant triangles whose vertices are rounded to integers, an algorithm likewise in scope here (Abrash (1997), chapter 69).

<sup>9</sup>Antialiasing “**didn’t appear in home console graphics architectures** until the debut of the [Nintendo 64] in late 1996”. McCornack et al. (1995) considers antialiasing among the features “unlikely” to be needed in game programming. <https://imagequalitymatters.blogspot.com/2011/01/retro-tech-analysis-virtua-racing-md-vs.html>

<sup>10</sup>Perspective correction accounts for distance from the viewer: closer objects appear larger. The lack of perspective correction (as in what is called *affine texture mapping*), together with the lack of smoothing (antialiasing) of edges, contributed to the characteristic distortion and instability of 3-D graphics in many video games for the original PlayStation.

<sup>11</sup>It is being considered whether to replace these 2-D limits with one of the following alternatives: 1. Instead of tiles, sprites, and layers, the game uses a *frame buffer* (array of color samples, called pixels, in computer memory) with no more than 8 bits per pixel (no more than 256 simultaneous colors) and all visuals in the game must be *rendered in software*. But I don’t know of a way to describe further restrictions useful for game programming in the mid- to late 1990s style. 2. The 2-D limits in the specification apply, but instead of replacing a 2-D layer, the 3-D layer is simply a special sprite that covers the game screen (the usual size limits for sprites don’t apply) and can have transparent and translucent pixels. 3. Same as (2), but in addition, there are no tiles or 2-D layers (all the graphics are sprites). The tile-based limits in this specification also suit games that support only text display, and thus have visuals that resemble the text modes (as opposed to graphics modes) found in PCs and computer terminals.

<sup>12</sup>The Neo Geo (1990) has only one 2-D layer; the rest of the visuals are sprites drawn below that layer.

<sup>13</sup>Translucent pixels enable *alpha blending* techniques (the mixing of one image with another). But alpha blending was

5. The 3-D layer’s dimensions, in pixels, are the same as the “screen resolution” (see later).
2. Tiles. A *tile* is a small rectangular array of pixels.
  1. Every tile has the same width and the same height as every other. The width must be 32 or less, and the height must be 32 or less.
  2. The application chooses one:
    1. Each tile is in a 1-bit-per-pixel format and uses one of 16 *color tables*, with 2 colors per table.
    2. Each tile is in a 2-bit-per-pixel format and uses one of 16 color tables, with 4 colors per table.
    3. Each tile is in a 4-bit-per-pixel format and uses one of 16 color tables, with 16 colors per table.
    4. There is a single 256-color table for use by tiles. Each tile is in an 8-bit-per-pixel format.
  3. Each color in each color table used by tiles is of the vertex color format given earlier.
  4. Tiles may contain transparent, but not translucent, pixels.
  5. Image files used by the game should not store “pre-pixelated” tiles.
  6. When referenced in a 2-D layer, a tile can be horizontally flipped, vertically flipped, or both.
3. Sprites. A *sprite* is a rectangular array of either tiles or pixels.
  1. Each sprite’s maximum width, in pixels, is determined as follows: Take the larger of the screen width and screen height, then divide by 4, then round up to the nearest power of 2. A sprite’s maximum height is the same as the maximum sprite width. (An alternative is that each sprite is no more than  $64 \times 64$  pixels in size.)
  2. Besides the previous point, sprites can have any width and height.
  3. Each sprite made of pixels (rather than tiles) has a pixel format allowed for *3-D textures*, given earlier, and may contain transparent, but not translucent, pixels.
  4. Each sprite can be drawn above or below any of the 2-D or 3-D layers.
  5. The application chooses one:
    1. Each sprite can undergo a 2-D affine transformation.
    2. Each sprite can be horizontally flipped, vertically flipped, or both.<sup>14</sup>
    3. No affine transformation or flipping of sprites is allowed.
  6. Image files used by the game should not store “pre-pixelated” sprites.
  7. Up to N sprites can be displayed at a time, where N is calculated as  $(\text{screen width} \times \text{screen height} \times 16) / (X \times Y)$ , rounded up, but not more than 512. In this formula, X is the maximum sprite width and Y is the maximum sprite height.<sup>15</sup>

**Note:** The suggested width and height for tiles is 8 pixels  $\times$  8 pixels.

### Examples:

1. For a “screen resolution” (see later) of  $640 \times 480$  pixels, one choice is: 4-bit-per-pixel tiles,  $8 \times 8$  tiles, sprites up to  $160 \times 160$  pixels, no more than 192 sprites at a time, and no flipping or transformation of sprites.
2. Here is what the maximum primitive counts look like for different screen resolutions (rounding down):  $256 \times 192 / 24 = 2048$ ;  $320 \times 200 / 24 = 2666$ ;  $320 \times 240 / 24 = 3200$ ;  $480 \times 360 / 24 = 6400$ ;  $640 \times 400 / 24 = 10666$ ;  $640 \times 480 / 24 = 12800$ .

---

“relatively new to PC games” at the time of *Quake*’s launch in 1996, according to Abrash (1997), and is practically not discussed at all in McCornack et al. (1995). Only images with opaque and/or transparent pixels tended to be supported in early-1990s video games.

<sup>14</sup>SEGA arcade machines from the 1980s and earlier had rudimentary systems for scaling (stretching or shrinking) sprites horizontally and vertically. In the Super Famicom/Super Nintendo Entertainment System (1990), sprites could not be scaled, but they could be flipped.

<sup>15</sup>Tile- and sprite-based graphics were in place largely because they saved memory; they were popularized by the arcade game *Galaxian* (1979). Indeed, this system, present in the Nintendo DS and many earlier game consoles, was abandoned in the Nintendo 3DS in favor of a frame buffer (see the “Architecture of Consoles” series by Rodrigo Copetti). Game consoles employing tile-based visuals tended to limit not only the number of sprites per frame, but also the number of sprites per row of pixels, but a per-row limit is not adopted here. As for the per-frame limit, the Famicom/Nintendo Entertainment System (1983), SEGA Master System/SEGA Mark III (1985), and PC Engine/TurboGrafx 16 (1987) had a limit of 64 sprites; the Game Boy (1989), 40; the SEGA Mega Drive/Genesis (1988), 80; the Super Famicom/Super Nintendo Entertainment System (1990), 128; and the relatively expensive Neo Geo (1990), 381. Again, see the “Architecture of Consoles” series by Rodrigo Copetti.

Other requirements:

- **Screen resolution:** The game screen image has no more than 307,200 total pixels (for example,  $640 \times 480$ , or 640 pixels horizontally and 480 pixels vertically).<sup>16</sup> Support for game screen resolutions larger than this limit, in addition to resolutions meeting the limit, is optional.
- **Rendering in software:** The game should include a mode in which the graphics are *rendered in software*. This means that their rendering does not rely on a video card, a visual accelerator chip, or the operating system’s graphics API (such as GDI, OpenGL, or Direct3D) with the sole exception of sending a finished game screen image to the player’s display (such as through GDI’s **StretchDIBits** or copying to VGA’s video memory). The game can optionally support hardware acceleration of graphics as well (and can even use such acceleration by default when the game detects its availability).
- **Music:** Music is in Standard MIDI files (SMF) only. The General MIDI System level 1 should be followed for such files.<sup>17</sup>

## 2.2 Classic Graphics in Scope

This specification for “classic graphics”<sup>18</sup> in modern games largely reflects the visual limitations of—

- consumer PCs (personal computers) released in the mid- to late 1990s,
- home computers released before 1995,<sup>19</sup>
- game consoles (handheld and for TVs) released before 2000,
- learning toys with an electronic display, released before 2000,
- arcade machines with similar performance to machines described earlier, and
- the Game Boy Advance and Nintendo DS, both of which were released after 2000 but have relatively meager graphical ability.

In addition, video-game graphics for personal digital assistants, graphical calculators, and cellular phones (generally those released before 2007) are within the spirit of this specification, up to the performance of

---

<sup>16</sup>If the game screen image uses two colors only (such as black and white), the game could choose to allow it to have up to 800,000 total pixels. For example, a  $1024 \times 768$  display has 786,432 total pixels. However, two-color graphical display modes larger than 307,200 total pixels are probably rare among consumers. The modern game *Return of the Obra Dinn* employs a two-color  $800 \times 450$  display (378,000 total pixels) (but even so this resolution was “up from  $640 \times 350$ ”). In the Godot engine, the screen resolution corresponds to the “Viewport Width” (**window/size/viewport\_width**) and “Viewport Height” (**window/size/viewport\_height**) project settings. For the Unity engine, there is advice from 2019 relating to the visual style in “8-bit” and “16-bit” game consoles. In Unreal Engine, the screen resolution apparently corresponds to **ResolutionSizeX** and **ResolutionSizeY**. But a lighter-weight graphics engine than Unity, Unreal, or even Godot would better suit the spirit of this specification. <https://blog.unity.com/technology/2d-pixel-perfect-how-to-set-up-your-unity-project-for-retro-8-bits-games> <https://blog.unity.com/technology/2d-pixel-perfect-how-to-set-up-your-unity-project-for-retro-16-bit-games>

<sup>17</sup>Standard MIDI files should be played back using a cross-platform open-source software synthesizer (see section “Building a Public-Domain music synthesis library and instrument banks”), using either FM or wave-table synthesis; most modern PCs no longer come with hardware synthesizers. I note that it’s possible to write an FM software synthesizer supporting every MIDI instrument in less than 1024 kibibytes of code. Standard MIDI files organize MIDI commands into up to 16 *channels*, each occupied by at most one “instrument” at a time. Under the *Multimedia PC Specification* (1992), the first ten channels were intended for high-end synthesizers (where the tenth is percussion); the thirteenth through sixteenth, for low-end ones (sixteenth is percussion), and the nonpercussion channels were arranged in decreasing order of importance. This convention was abandoned with the rise in support for the General MIDI System level 1 (see Q141087, “DOCERR: MarkMIDI Utility Not Provided in Win32 SDK”, in the Microsoft Knowledge Base): now all 16 channels are supported (with only the tenth for percussion) and need not be arranged by importance.

<sup>18</sup>Matt Saettler, “Graphics Design and Optimization”, Multimedia Technical Note (Microsoft), 1992, contains a rich discussion of graphics used in computer games and other audiovisual computer applications up to 1992. Not mentioned in that document are graphics resembling: (1) Segmented liquid crystal displays, of the kind found in Nintendo’s Game & Watch and many Tiger Electronics handheld games. These are simple to emulate, though: design a screen-size image that assigns each segment a unique color and, each frame, draw black where the segments that are “on” are, and draw white (or another background) elsewhere on the screen. (2) Vacuum fluorescent displays, notable in user interfaces of some media player applications that resemble a “stereo rack system”.

<sup>19</sup>Home computers include IBM PC compatibles, PC-88 and PC-98 families, ZX-Spectrum, Atari ST family, MSX family, Commodore 64, Amiga family, Commodore PET and VIC-20, BBC Micro, Amstrad CPC, Acorn Archimedes, Tandy TRS-80, Apple II, and computers running MS-DOS, Windows (up to Windows 98), Macintosh operating system (Mac OS) up to 9.x, NeXTSTEP, OS/2, or X Window System.

consumer PCs released before 2000.<sup>20</sup>

Arcade games in scope range in year from 1971 — which saw the launch of the first arcade video game, *Computer Space* — to 1999. But some arcade machines, such as SEGA Model 3 (1996) and SEGA NAOMI (1998), may have 3-D rendering capabilities beyond what is “classic” here.

In general, PC applications that feature classic graphics include:

1. Windows applications written for DirectX versions earlier than 7 and using Direct3D or DirectDraw for rendering visuals.
2. Windows games using GDI or **WinG**<sup>21</sup> for graphics and supporting Windows 98 or earlier. Examples are *Chip’s Challenge* for Windows (1992) and Brian Goble’s *The Adventures of MicroMan* (1993).
3. Games for MS-DOS or PC-9801 that were published before 2000. Examples are *Quake* (1996), *WarCraft* (1994), and the first titles of the Touhou Project series (1997-1998).
4. Games using OpenGL 1.2 or earlier for graphics.
5. So-called “multimedia titles” from the 1990s, or applications resembling interactive versions of books (generally reference and other nonfiction works), complete with sound, animation, and video. See the *Authoring Guide* that came with Microsoft’s Multimedia Development Kit.

NVIDIA GeForce 256 (late 1999) is out of scope.

One of the following games can be considered an upper limit to what is considered “classic graphics” in this specification.

- *Quake III Arena* (December 1999), which **required DirectX 7 and at least 64 million bytes of memory**<sup>22</sup>.
- *Falcon 4.0* (1998).

## 2.3 Optional Limits

A game may impose further constraints to this specification (for example, to reduce the maximum number of 3-D triangles, to disallow 3-D graphics, to reduce the number of colors per tile allowed, or to **reduce to a limited set the colors**<sup>23</sup> ultimately displayed on screen). I would be interested in knowing about these limitations that a new game that adopts this document decides to impose.

Examples of optional constraints are the following:

- The game displays no more than 16 colors at a time.<sup>24</sup>
- The game is limited to the 16 colors of the so-called *VGA palette*.
  - In the 8-bit-per-component color format, this palette’s colors are: light gray, that is, (192, 192, 192); or each color component is 0 or 255; or each color component is 0 or 128.
  - In the vertex color format, the closest colors to this palette are: 24/24/24; or each color component is 0 or 16; or each color component is 0 or 31.
- The game displays no more than 256 colors at a time.<sup>25</sup>
- The game is limited to a fixed set of 254 colors.
- All game files can be packaged in a ZIP file or Win32 program file that takes no more than—

<sup>20</sup>Examples are the **Sharp MI-Zaurus** (2000) and the many cellular phones that came with Java Micro Edition, its Mobile Information Device Profile (MIDP, **Java specification request 37** and **JSR 118**), and extensions (especially **JSR 184**, **Mobile 3D Graphics API**). [https://dench.flatlib.jp/app/chiraks\\_em](https://dench.flatlib.jp/app/chiraks_em) <https://jcp.org/en/jsr/detail?id=37> <https://jcp.org/en/jsr/detail?id=118> <https://jcp.org/en/jsr/detail?id=184>

<sup>21</sup>[https://www.pcgamingwiki.com/wiki/List\\_of\\_WinG\\_games](https://www.pcgamingwiki.com/wiki/List_of_WinG_games)

<sup>22</sup>[https://www.dosdays.co.uk/topics/early\\_3d\\_games.php](https://www.dosdays.co.uk/topics/early_3d_games.php)

<sup>23</sup>[https://peteroupc.github.io/classic-wallpaper#Color\\_Palettes](https://peteroupc.github.io/classic-wallpaper#Color_Palettes)

<sup>24</sup>An example is *Loom* (1990).

<sup>25</sup>This was recommended for Macintosh games in McCornack et al. (1995), notably because 8-bit-per-pixel images transfer faster and save memory over images with more bits per pixel; see also chapter 70 of Abrash (1997) (dealing with the porting of an 8-bit-per-pixel game to a 16-bit-per-pixel video card).

- 1,457,664 bytes (the capacity of a file-allocation-table (FAT) formatted high-density 3.5-inch floppy disk), or
- 1,213,952 bytes (the capacity of a FAT formatted high-density 5.25-inch floppy disk), or
- 730,112 bytes (the capacity of a FAT formatted normal-density 3.5-inch floppy disk), or
- 362,496 bytes (the capacity of a FAT formatted double-density 5.25-inch floppy disk), or
- 65,536 bytes,<sup>26</sup> or
- 681 million bytes (slightly less than the maximum total size of files in a formatted CD-ROM).
- The game uses no more than 16 million bytes of system memory at a time.
- The game uses no more than 655,360 bytes of system memory (plus 262,144 bytes of additional memory for graphical use only) at a time.<sup>27</sup>
- The game is a Win32 application compatible with Windows XP.
- The game is a Win32 application compatible with Windows 98.
- The game aims for a rate of 30 frames per second.
- The game’s graphics must be *rendered in software*.
- The game’s rendering of visuals employs only 32-bit and smaller integers, integer arithmetic, and fixed-point arithmetic.<sup>28</sup>
- The game’s rendering of visuals employs only 32-bit and smaller integers, integer arithmetic, fixed-point arithmetic, and arithmetic involving 32-bit floating-point numbers (IEEE 754 binary32, or the commonly implemented meaning of the C language’s `float` type).<sup>29</sup>
- The game renders only white line segments on a black background (or vice versa), and displays no more than 320 of those segments at a time. These line segments are drawn with an approximation at integer coordinates.
- The game runs with acceptable performance on **80486-compatible processors**<sup>30</sup> at 33 or 50 megahertz.<sup>31</sup>
- The game runs with acceptable performance on x86 processors at 100 megahertz.

## 2.4 Notes on Specification

This section has notes on this specification, such as how its requirements correspond to the graphics abilities of pre-2000 video games.

### 2.4.1 Screen resolutions

- Screen resolutions larger than 307,200 total pixels (such as  $800 \times 600$ ) are not within the spirit of this challenge, even though more demanding games in the late 1990s, as well as the *PC 98 System Design Guide* (1997), aimed for the  $800 \times 600$  resolution or higher for 3-D graphics. Indeed, for the most part, major game consoles and arcade machines in the 1990s and earlier supported a resolution of no more than 307,200 total pixels.<sup>32</sup>

<sup>26</sup>Popular file size limit of so-called “64K intros”.

<sup>27</sup>MS-DOS applications are normally limited to 640 kibibytes or less of *conventional memory*, along with whatever memory is carried by the video card. (PCs running on the very early Intel 8086 and 8088 processors can map out no more than 640 kibibytes of system memory.) 262,144 bytes is the usual minimum of graphics memory for VGA video cards.

<sup>28</sup>It wasn’t until the Pentium processor’s advent that floating-point arithmetic was embraced in 3-D game programming: for example, see chapter 63 of Abrash (1997).

<sup>29</sup>The use of 64-bit floating-point numbers (IEEE 754 binary64, or the commonly implemented meaning of the C language’s `double` type) is perhaps unusual in graphics programming before 2000.

<sup>30</sup><https://www.dosdays.co.uk/topics/cpus.php#i486>

<sup>31</sup>This is the performance level targeted for the “**CyberVGA**” engine by Burak Yazar. <https://expfunction.itch.io/cybervga>

<sup>32</sup>Moreover, PC games before 2000 that required screen resolutions larger than  $640 \times 480$  are rare, and according to PCGamingWiki they include the following games (most of which are 2-D): *Timon & Pumbaa’s Jungle Games* (1995); *Tequila & Boom Boom* (1995); *Romance of the Three Kingdoms IV: Wall of Fire* (1995/1996); *Joint Strike Fighter* (1997), but only when run with the Glide programming interface; *Links LS: 1998 Edition* (1997); *Emergency: Fighters for Life* (1998); *Championship Manager: Season 99/00* (1999); *Heroes of Might and Magic III* (1999); *Alien Nations* (1999); *Pizza Syndicate/Fast Food Tycoon* (1999); *Age of Empires II: The Age of Kings* (1999).

- Screen resolutions that have been used in classic games include:<sup>33</sup>
  - Video graphics array (VGA) display modes:  $640 \times 480$ ,<sup>34</sup>  $320 \times 240$ ,<sup>35</sup>  $320 \times 200$ .<sup>36</sup>
  - 4:3 aspect ratio:  $640 \times 480$ ,<sup>37</sup>  $512 \times 384$ ,<sup>38</sup>  $400 \times 300$ ,<sup>39</sup>  $320 \times 240$ ,<sup>40</sup>  $256 \times 192$ ,<sup>41</sup>  $160 \times 120$ .<sup>42</sup>
  - Game console aspect ratios:  $640 \times 448$ ,<sup>43</sup>  $320 \times 224$ ,<sup>44</sup>  $256 \times 224$ ,<sup>45</sup>  $256 \times 240$ ,<sup>46</sup>  $240 \times 160$ ,<sup>47</sup>  $160 \times 144$ .<sup>48</sup>
  - 5:4 aspect ratio:<sup>49</sup>  $320 \times 256$ ,<sup>50</sup>  $360 \times 288$ .<sup>51</sup>
  - Two-color graphics:  $720 \times 348$ ,<sup>52</sup>  $640 \times 200$ ,<sup>53</sup>  $512 \times 342$ .<sup>54</sup>
  - Enhanced Graphics Adapter aspect ratio:  $640 \times 350$ .<sup>55</sup>
  - 8:5 aspect ratio:  $640 \times 400$ ,<sup>56</sup>  $320 \times 200$ .<sup>57</sup>
  - Other:  $280 \times 192$ ,<sup>58</sup>  $480 \times 272$ ,<sup>59</sup>  $512 \times 424$ ,<sup>60</sup>  $400 \times 240$ ,<sup>61</sup>  $384 \times 224$ ,<sup>62</sup>  $160 \times 200$ ,<sup>63</sup>  $480 \times 240$ .<sup>64</sup>

This is not a complete list. Some game consoles, such as the SEGA Saturn or Nintendo 64, allowed games to alter the screen resolution during gameplay. **Arcade game graphics** are described later.

- As of early 1997, “[s]urveys indicate[d] that the great majority of [PC] users operate[d] in 640[ $\times$ ]480 resolution with 256 colors”.<sup>65</sup>

---

<sup>33</sup>In addition to the resolutions shown here, there are modern games that employ low resolutions with the same 16:9 aspect ratio as high-definition displays. These include  $640 \times 360$  (*Blasphemous* [2019]);  $400 \times 225$  (*Unsighted*);  $480 \times 270$  (*Enter the Gungeon*);  $320 \times 180$  (*Celeste*). Information about game consoles’ screen resolution can be found in Rodrigo Copetti’s “Architecture of Consoles” series.

<sup>34</sup>VGA mode 12h (16 colors).

<sup>35</sup>PlayStation (One); Nintendo 3DS lower screen; larger VGA “mode X” (256 colors).

<sup>36</sup>Commodore 64; NEC PC-8001; VGA mode 13h (256 colors), especially seen in MS-DOS games; Color/Graphics Adapter (CGA) 4-color mode; Atari ST 16-color mode; **Amiga NTSC**. <https://blog.johnnovak.net/2022/04/15/achieving-period-correct-graphics-in-personal-computer-emulators-part-1-the-amiga>

<sup>37</sup>VGA mode 12h (16 colors).

<sup>38</sup>One commonly supported “super-VGA” mode, especially in mid-1990s gaming, and which was also recommended by the *PC 98 System Design Guide*.

<sup>39</sup>One low resolution recommended by the *PC 98 System Design Guide*.

<sup>40</sup>PlayStation (One); Nintendo 3DS lower screen; larger VGA “mode X” (256 colors).

<sup>41</sup>Nintendo DS; NEC PC-6001; SEGA Master System/SEGA Mark III; MSX; Colecovision.

<sup>42</sup>Rarely used VGA display mode.

<sup>43</sup>PlayStation 2 NTSC.

<sup>44</sup>SEGA Mega Drive/SEGA Genesis; Neo Geo NTSC.

<sup>45</sup>Effective resolution of Famicom/Nintendo Entertainment System NTSC; Super Famicom/Super Nintendo Entertainment System NTSC; minimum resolution of PC Engine/TurboGrafx 16.

<sup>46</sup>Nintendo Entertainment System PAL; Super Nintendo Entertainment System PAL.

<sup>47</sup>Game Boy Advance.

<sup>48</sup>Game Boy, Game Boy Color, SEGA Game Gear.

<sup>49</sup>Aspect ratio found above all in PAL (phase-alternating-line) displays. The resolution  $640 \times 512$  (PlayStation 2 PAL), included in this category, covers more than 307,200 total pixels.

<sup>50</sup>Amiga PAL (same pixel spacing horizontally as vertically); Neo Geo PAL.

<sup>51</sup>PAL overscan.

<sup>52</sup>Hercules Graphics Card two-color.

<sup>53</sup>Color/Graphics Adapter (CGA) two-color; NEC PC-8801 8-color mode; Atari ST 4-color mode.

<sup>54</sup>12-inch classic Macintosh.

<sup>55</sup>16 colors.

<sup>56</sup>NEC PC-9801 8-color mode; Atari ST two-color.

<sup>57</sup>Commodore 64; NEC PC-8001; VGA mode 13h (256 colors), especially seen in MS-DOS games; Color/Graphics Adapter (CGA) 4-color mode; Atari ST 16-color mode; **Amiga NTSC**. <https://blog.johnnovak.net/2022/04/15/achieving-period-correct-graphics-in-personal-computer-emulators-part-1-the-amiga>

<sup>58</sup>Apple II.

<sup>59</sup>PlayStation Portable.

<sup>60</sup>MSX 2.

<sup>61</sup>Effective resolution of Nintendo 3DS upper screen without parallax effect.

<sup>62</sup>Virtual Boy.

<sup>63</sup>One “Tandy graphics adapter” mode.

<sup>64</sup>Minimum resolution for “handheld PCs” (*Windows CE Programmer’s Guide*, MSDN Library, June 1998).

<sup>65</sup>S. Pruitt, “Frequently Asked Questions About HTML Coding for Internet Explorer 3.0”, updated Jan. 30, 1997.



- A game can support—

- multiple sizes for the area of the screen where the game’s action is drawn, or
- pixel-column or -row doubling as a “quality” setting,

or both features, without changing the size of the game’s image. For example, the original *Doom* (1993) supported several sizes of this kind (on PC, they were  $96 \times 48$ ,  $128 \times 64$ ,  $160 \times 80$ , and so on up to  $288 \times 144$ , as well as  $320 \times 168$  and  $320 \times 200$ ) and optional pixel-column doubling.<sup>66</sup>

- Games within the scope of this challenge are meant to be run in a desktop window if the player’s display is  $800 \times 600$  pixels or larger. The same is true if the game’s resolution is  $620 \times 420$  or smaller and the player’s display is  $640 \times 480$ . The game may also support full-screen display.

### 2.4.2 Frame rate

- No particular frame rate is required.<sup>67</sup>
- Modern games implementing this specification can choose to target a frame rate typical of today, such as 30, 40, or 60 frames per second.
- Game consoles for TVs were designed for how often TVs can draw their image (nearly 60 frames per second for NTSC<sup>68</sup> and 50 for PAL<sup>69</sup>).
- *Doom* (1993) operated at 35 frames per second but could not be run at that rate (under default settings) by typical PCs of the time.<sup>70</sup>
- For comfort reasons, a minimum frame rate may be required for video games that offer “**3-D vision**”<sup>71</sup> by rendering multiple views of the scene at a time, in conjunction with special glasses (for example, a SEGA Master System accessory) or a virtual-reality headset (for example, Nintendo’s Virtual Boy). But such games were rare before 2000.

### 2.4.3 3-D graphics

- The *PC 99 System Design Guide*<sup>72</sup> sections 14.27 to 14.34 gives guidelines on 3-D graphics support for PCs to be launched in 1999. This challenge recommends the writing of game software with graphics performance as good as hardware meeting such guidelines, except for the screen resolution, frame rate, and double buffering requirements.
- An application may choose to support stencil buffers, bump mapping, environment mapping, and three- or four-texture blending, but these are borderline pre-2000 graphics capabilities.
- For years earlier than 1999, some of the 3-D capabilities mentioned in the specification (such as texture blending) might not be typical.
- This specification allows for:

<sup>66</sup>Fabien Sanglard, *Game Engine Black Book: Doom*.

<sup>67</sup>Until the early 1990s, the number of color samples (pixels) an application can transfer per second was usually small, limiting the supported size and frame rate for arbitrary video content. Indeed, for example, the *Multimedia PC Specification* (1992) recommended that video cards be able to transfer up to 8-bit-per-sample graphics at a rate of 140,000 samples per second or faster given 40 percent of CPU bandwidth. The Multimedia PC level 2 specification (1993) upped this recommendation to 1.2 million samples per second (sufficient for  $320 \times 240$  video at 15 frames per second, the recommendation in article Q139826, “AVI Video Authoring Tips & Compression Options Dialog Box”, 1995). For details on these specifications, see article Q106055 in the Microsoft Knowledge Base. Both recommendations are far from the 6.144 million samples per second needed to display  $640 \times 480$  video smoothly at 20 frames per second.

<sup>68</sup>Stands for the National Television Standards Committee of the Electronics Industries Association. “NTSC” often refers to the video standard known as RS-170A.

<sup>69</sup>Stands for phase alternating line.

<sup>70</sup>Fabien Sanglard, *Game Engine Black Book: Doom*.

<sup>71</sup>[https://www.pcgamingwiki.com/wiki/Glossary:Native\\_3D](https://www.pcgamingwiki.com/wiki/Glossary:Native_3D)

<sup>72</sup>Intel & Microsoft, *PC 99 System Design Guide*, 1998. ISBN 0-7356-0518-1.

- Prerendered graphics (as in *Space Quest 5*, *Myst*, or the original *Final Fantasy VII* on PlayStation [1997]), to simulate showing highly detailed imagery.
- Drawing a 3-D graphic as a *voxel mesh*<sup>73</sup> (formed from point samples in 3-D, rather than 2-D, called *voxels*), as long as the triangle limits are respected.
- The following are not within the spirit of this challenge:
  - Displaying more than 20,000 triangles at a time (per frame), even for higher screen resolutions. Most 3-D video games before 2000 displayed well fewer than that, but there may be exceptions, such as arcade games for the SEGA Model 3.
  - Phong shading (per-pixel specular highlighting), ray tracing, and path tracing, which were too slow for real-time use in the 20th century. Exceptions include (1) two-dimensional (per-screen-column) *ray casting*, and (2) *light maps* with precomputed lighting.
- It wasn't until 1995 that 3-D video cards became widely available for consumer PCs.<sup>74</sup> In 3-D video games for PCs “[i]n 1995/1996, it was not uncommon to have 30-50% of the game screen filled with polygons without textures” (according to an **article**<sup>75</sup> that compared *Havoc* [1995] with *Mortal Kombat 4* [1997]).
- This specification is not centered on video games that offer “3-D vision” (see note under “Frame rate”), given how rare they were before 2000.

#### 2.4.4 Arcade game graphics

- From 1971 to 1979, arcade games tended to have white visuals drawn on a black screen; the screen image was one bit per pixel. Some arcade cabinets in the late 1970s had a colored semitransparent overlay in front of the screen to lend color to the otherwise white graphics (an example is *Lunar Rescue* [1979]).
- The practice of drawing graphics on a mostly black screen continued up to the early 1980s, but this time they were colored by the game itself rather than by an overlay.
- The visuals of some arcade games from 1979 to the early 1980s were made of white line segments. Notable examples are *Asteroids* (1979) and *Battlezone* (1980). In *Star Wars* (1983), these line segments were even multicolored.
- Graphics made of tiles and sprites were popularized by the arcade game *Galaxian* (1979).
- Most arcade games before 1995 had a resolution from 46,080 pixels ( $192 \times 240$  or  $240 \times 192$ ) through 98,304 pixels ( $256 \times 384$  or  $384 \times 256$ ). The following were exceptions:
  - Certain two- or three-screen games by Taito (for example, *Darius* [1987] and *The Ninja Warriors* [1987]).
  - The Midway MCR family of arcade machines (1980 to about 1986) had a screen resolution of  $480 \times 480$ ,  $512 \times 448$ , or  $512 \times 480$ , but backgrounds were drawn at half the resolution of sprites. Examples include *Tapper* (1982) and *Wacko* (1983).
- If an arcade game employed 2-D visuals with tiles and sprites, each tile and sprite tended to use no more than 16 colors, and semitransparency was not used.

#### 2.4.5 Screen image effects (filters)

- Effects that modify the game screen image to emulate CRT displays<sup>76</sup> are outside the scope of this challenge. So are effects that **scale**<sup>77</sup> the game screen to fit the height or width of the player's display.<sup>78</sup>

<sup>73</sup><https://blog.danielschroeder.me/blog/voxel-renderer-objects-and-animation>

<sup>74</sup>By contrast, 3-D video cards have been offered for professional-use computers since the mid-1980s; the first such cards for PCs that supported real-time display were **introduced in 1988**. <https://retro.swarm.cz/sgi-irisvision-add-in-3d-accelerator-for-pc-1990/>

<sup>75</sup><https://retro.swarm.cz/s3-virge-325-vx-dx-gx-gx2-series-of-early-3d-accelerators-deep-dive/>

<sup>76</sup>CRT displays, or cathode-ray-tube displays, were the typical kind of computer monitors and TVs in the 1980s and 1990s.

<sup>77</sup><https://www.pcgamingwiki.com/wiki/Glossary:Scaling>

<sup>78</sup>Effects to scale the game screen include so-called “pixel-art scaling algorithms” such as **HQX** and **2xSaI**, as well as bilinear or point filtering. Effects to scale the game screen do not include the decoding of small videos to fit the *game screen*, as opposed to the player's display. It was common for 1990s games to have videos smaller than the game screen and to scale

This specification assumes those effects are not in place. A game can have those effects if it wishes, but they should be in-game settings.

## 2.4.6 Sounds

- Besides the limitation on music, this specification has no further limitations on sounds.
- Early game consoles supported sound only through one or more *programmable sound generators*, such as square and triangle wave generators, as opposed to digitized sounds<sup>79</sup>. Games that choose to constrain file size may wish to implement software versions of programmable sound generators for at least some of their sounds.
- When digitized sounds are supported in classic games, they typically have a sample rate of 8000, 11,025, 22,050, or 44,100 Hz, are either mono or stereo, and take 8 or 16 bits per sample.<sup>80</sup>

## 2.4.7 Memory

- This specification does not impose a limit on graphical memory use (akin to the video memory, or VRAM, of a video card). Such a limit is optional. One suggested example, given in kibibytes, is the screen width times screen height divided by 24, which is slightly less than 13.2 million bytes for  $640 \times 480$  resolution. (A kibibyte is 1024 bytes.) Imposing a limit on graphical memory use does not limit the size or number of textures, 3-D models, images, or other resource files a game can have.<sup>81</sup>
- According to “**Typical PCs Each Year**”<sup>82</sup>, the following ranges of system memory were typical for PCs sold in the specified years:<sup>83</sup>
  - 1994: 4MB to 8 MB, with more expensive PCs having 16 MB.<sup>84</sup>
  - 1997: 8MB to 32MB.<sup>85</sup>
  - 1998: 32MB to 128MB.<sup>86</sup>

## 2.5 Seeking Comments

As with the rest of this open-source article, **comments on this specification**<sup>87</sup> are welcome. But most useful would be comments that improve or refine the specification to fit the graphical abilities of pre-2000 video games.

Examples are comments that give *measurements* (or references to other works that make such measurements) on the graphical performance actually achieved, in normal gameplay, by video games released in 1999 and earlier (or released in, say, 1994 or earlier) for home computers or game consoles. (It bears repeating: *measurements*, not inferences or guesses from screenshots or videos.)

This includes statements like the following, with references or measurements:

- “Game X shows up to Y polygons at a time at Z frames per second and screen resolution W”.

---

those videos to fit the game screen “on the fly”, in the process of displaying them. For example, such a game could decode videos of size 160x100 to fit a game screen of  $320 \times 200$ . (See, for instance, Nigel Thompson, “**Stretching 256-Color Images Using Interpolation**”, Microsoft Developer Network, March 7, 1995.) [[https://learn.microsoft.com/en-us/previous-versions/ms969922\(v=msdn.10\)](https://learn.microsoft.com/en-us/previous-versions/ms969922(v=msdn.10))]([https://learn.microsoft.com/en-us/previous-versions/ms969922\(v=msdn.10\)](https://learn.microsoft.com/en-us/previous-versions/ms969922(v=msdn.10)))

<sup>79</sup>Sound today is most commonly digitized by pulse-code modulation (PCM), and PCM-digitized sound is often stored in computer files ending in “.WAV”.

<sup>80</sup>The *Multimedia PC Specification* (1992) required support in “multimedia PCs” for playback of at least 8-bit-per-sample mono digitized sound at 11,025 and 22,050 Hz. The Multimedia PC level 2 specification (1993) required support in “multimedia PCs” for playing back at least 16-bit-per-sample stereo digitized sound at 44,100 Hz.

<sup>81</sup>PC games released in 1999 tended to require 32 million bytes of system memory. Meanwhile, *Quake* (1996) required 8 million and recommended 16 million bytes of system memory.

<sup>82</sup>[https://www.dosdays.co.uk/topics/typical\\_pc\\_per\\_year.php](https://www.dosdays.co.uk/topics/typical_pc_per_year.php)

<sup>83</sup>“MB” is ambiguous here; it often means either one million bytes or 1024 times 1024 bytes.

<sup>84</sup>“Typical PCs in 1994”, <https://www.dosdays.co.uk/topics/1994.php>.

<sup>85</sup>“Typical PCs in 1997”, <https://www.dosdays.co.uk/topics/1997.php>.

<sup>86</sup>“Typical PCs in 1998”, <https://www.dosdays.co.uk/topics/1998.php>.

<sup>87</sup>[https://www.reddit.com/r/retrogamedev/comments/1rl36fo/pre2000\\_computer\\_graphics\\_for\\_modern\\_video\\_games/](https://www.reddit.com/r/retrogamedev/comments/1rl36fo/pre2000_computer_graphics_for_modern_video_games/)

- Statements like this that relate to polygons or triangles per frame are hard to find and often anecdotal, and they cannot always be inferred from screenshots or videos of gameplay. For example: (1) “A typical scene in a current [PC] application has 2000 to 2500 triangles per frame” (Fosner 1999)<sup>88</sup>. (2) “For context, *Quake* on a Pentium Pro pumped out maybe 100K triangles/second (tris/sec.) ... at best” (Abrash 2000)<sup>89</sup>; to be noted here is that the game normally ran at a screen resolution of  $320 \times 240$ . (3) According to the help for the 3DMark2000 benchmark, that benchmark comes with two game scenes that average up to 9,400 polygons in low detail and up to 55,000 in high detail. (4) The game engine for *SpecOps: Rangers Lead the Way* (1998) targeted 10,000 triangles per frame<sup>90</sup>. So did *Quake III Arena* (1999) (John Carmack .plan, Sep. 2, 1999).
- “Scenes in game X have Y triangles on average”.
- “Game X shows no more than [16 or 256] simultaneous colors”.
- “Game X uses Y bytes of memory while running on Windows 98”.
- “Game X shows up to Y sprites at a time [at screen resolution Z]” (for 2-D games such as those built using the tool Director, then by Macromedia).
- The 2-D game X, from year Y, supports a **given 2-D graphics capability**<sup>91</sup> (for example, 2-D rotations of sprites; filling ellipses with a solid color; flood filling; antialiasing of lines and shapes; translucent alpha blending; translucent sprites).
- The 3-D game X, from year Y, supports a **given 3-D graphics capability**<sup>92</sup> (for example, texture mapping, Gouraud shading, bump mapping, edge antialiasing, alpha blending, texturing of most polygons in a scene, or MIP mapping).

(Those statements will also help me define constraints for video games up to an earlier year than 1999.)

Statements like the following are also useful, with references:

- “In year X [1999 or earlier], Y% of PC users used screen resolution Z”.
- In year X, a given 3-D graphics capability became typical in 3-D video games.
- In year X, a given 2-D graphics capability became typical in 2-D video games.
- “In year X [1999 or earlier], Y% of home computers in use were equipped with Z million bytes of memory”.
- “In year X, Y% of home PCs were equipped with 3-D video cards”.
- A market-share-weighted average of system memory requirements of video games in year X.
- On a market-share-weighted basis, X% of video games in year Y—
  - ran on [16 or 256]-color display modes, or
  - were 3-D video games, or
  - were played on arcade machines, or
  - were played on PCs, or
  - were played on game console Z.

By contrast, statements like the following are not very useful, since they often don’t relate to the actual performance of specific video games:

- “Game console X can process up to Y triangles per second”.
- “Video card X can render up to Y polygons per frame”.
- “Video card X can render up to Y pixels per second”.
- “Game X renders Y triangles per second”, without stating the frame rate or the screen resolution.
- “Game X issues Y draw calls per frame”, since a single draw call can draw one triangle or tens of thousands.

<sup>88</sup>R. Fosner, “DirectX 6.0 Goes Ballistic With Multiple New Features And Much Faster Code”, *Microsoft Systems Journal* January 1999.

<sup>89</sup>M. Abrash, “Inside Xbox Graphics”, *Dr. Dobbs’s Journal*, August 2000.

<sup>90</sup>“Postmortem: Zombie’s *SpecOps: Rangers Lead the Way*”, Jan. 31, 2000. <https://www.gamedeveloper.com/design/postmortem-zombie-s-i-specops-rangers-lead-the-way-i->

<sup>91</sup>[https://peteroupc.github.io/graphicsapi.html#2\\_D\\_Graphics](https://peteroupc.github.io/graphicsapi.html#2_D_Graphics)

<sup>92</sup>[https://peteroupc.github.io/graphicsapi.html#3\\_D\\_Graphics](https://peteroupc.github.io/graphicsapi.html#3_D_Graphics)

- “Character models in game X average Y triangles”.

The following are examples of the kind of statements desired:

- *Actua Soccer (VR Soccer '96)* (1995) **averaged 776 triangles per frame** at  $640 \times 480$  resolution.
- *Terminal Velocity* (1995) **averaged 498 triangles per frame** at  $640 \times 480$  resolution.
- A benchmark of *Quake III Arena* averaged about 3,250 and topped out at about 6,970 triangles per frame after back-face culling, at screen resolution  $640 \times 480$  (Antochi et al. 2003)<sup>93</sup>, (Antochi et al. 2004)<sup>94</sup>.

## 2.6 Further Reading

- Abrash, M., *Michael Abrash's Graphics Programming Black Book: Special Edition*<sup>95</sup>, 1997.
- (Akenine-)Möller, T., Haines, E., *Real-Time Rendering* (first edition), 1999.
- Donnelly, P., “Moving Your Game to Windows, Part III: Sound, Graphics, Installation, and Documentation”, Microsoft Developer Network, Nov. 25, 1996.
- Houston, B., “Rendering Real-Time 3D Before GPUs”, Jul. 17, 2026. <https://ben3d.ca/blog/rendering-real-time-3d-before-gpus>
- Lamothe, A., *Tricks of the Game Programming Gurus*, Sams, 1994.
- Lamothe, A., *Black Art of 3D Game Programming*, Waite Group Press, 1995.
- Lamothe, A., *Tricks of the 3D Game Programming Gurus: Advanced 3D Graphics and Rasterization*, Sams, 2003. Published after 1999, but most of the 3-D capabilities discussed there are within the spirit of this specification.
- Lamothe, A., *Tricks of the Windows Game Programming Gurus*, Sams, 1999.
- J. McCornack et al., *Tricks of the Mac Game Programming Gurus*, Hayden Books, 1995.
- Roca, Jordi, et al., “**Workload Characterization of 3D Games**”<sup>96</sup>, *2006 IEEE International Symposium on Workload Characterization*. IEEE, 2006. Study on measuring certain features of 3-D games that are of interest in this specification, including triangles per frame. See the **attila-sim repository**<sup>97</sup>.
- Rodent, H., “Animation in Win32”, Microsoft Developer Network, Feb. 1, 1994.
- Thompson, N., *Animation Techniques in Win32*, Microsoft Press, 1995.

## 3 Building a Public-Domain music synthesis library and instrument banks

To improve support for MIDI (Musical Instrument Digital Interface) music playback in open-source and other applications, I challenge the community to write the following items, all of which must be released to the public domain or under the Unlicense.

- A cross-platform open-source library for *software* synthesis (translation into digitized sound) of MIDI data stored in standard MIDI files (SMF, .mid), using instrument sound banks (synthesizer banks) in SoundFont 2 (.sf2), Downloadable Sounds (.dls), and in OPL2, OPL3, and other FM synthesis sound banks, and possibly also in Timidity++/UltraSound patch format (.cfg, .pat). (Similar to *Fluidsynth*, but in the public domain or under the Unlicense. Instrument sound banks are files that describe how to render MIDI instruments as sound. In addition, the source code in the nonpublic-domain *foo\_midi*,

<sup>93</sup>Antochi, Iosif, et al. “3D Graphics Benchmarks for Low-Power Architectures.” 14th Annual Workshop on Circuits, Systems and Signal Processing. 2003.

<sup>94</sup>Antochi, Iosif, et al., “GaalBench: a 3D graphics benchmark suite for mobile phones”, *ACM SIGPLAN Notices* 39(7), 2004.

<sup>95</sup><https://github.com/jagregory/abrash-black-book>

<sup>96</sup><https://ieeexplore.ieee.org/abstract/document/4086130>

<sup>97</sup><https://github.com/attila-gpu/attila-sim>

*libADLMIDI*, *libOPNMIDI*, *OPL3BankEditor*, and *SpessaSynth* may be useful here, but review their licenses first.)

- The library should support popular loop-point conventions found in MIDI files.
- The library should support seeking of MIDI files such that a pause and resume function can be offered by a media player.
- An instrument sound bank for wave-table synthesis of all instruments and percussive (drum) noises in the General MIDI System level 1 specification.
  - Instruments should correspond as closely as possible to those in that specification, but should be small in file size or be algorithmically generated.
  - Instruments can be generated using the public-domain single-cycle wave forms found in the AdventureKid Wave Form collection, found at: **AKWF-FREE**<sup>98</sup>.
  - The samples for each instrument may, but need not, be generated by an algorithm, such as one that renders the instrument's tone in the frequency domain. An example of this is found in **com.sun.media.sound.EmergencySoundbank**<sup>99</sup>, which however is licensed under the GNU General Public License version 2 rather than public domain.
  - The instrument sound bank should be in either SoundFont 2 (.sf2) or Downloadable Sounds (.dls) format. See next section on a challenge to writing a guide on sound bank development.
  - The volume of all instruments in the sound bank should be normalized; some instruments should not sound louder than others.
- An instrument sound bank for FM synthesis of all instruments and percussive noises in the General MIDI System level 1 specification. Instruments should correspond as closely as possible to those in that specification.

## 4 Other Challenges and Projects

Other challenges and projects I make to the computer community.

None of the following creations should be generated by artificial-intelligence tools or with their help.

### 4.1 Classic desktop wallpaper

See the “**peteroupc/classic-wallpaper**”<sup>100</sup> repository for a challenge on creating tileable desktop wallpapers with a limited selection of colors and limited dimensions in pixels — such wallpapers are getting ever harder to find because desktop backgrounds today tend to cover the full computer screen, to employ thousands of colors, and to have a high-definition resolution (1920 × 1080 or larger).

### 4.2 Button and border styles for classic interfaces

See “**Traditional User-Interface Graphics**”<sup>101</sup> for a challenge on writing computer code (released to the public domain or under the Unlicense) to draw button and border styles for classic graphical user interfaces.

### 4.3 Sound bank development guide

Write an open-source and detailed guide on using free-of-cost software to produce decent-quality instrument banks from the recordings of real musical instruments (rather than copying or converting other instrument banks or recording from commercial synthesizers). See the section on **building instrument banks**, earlier. For this purpose, a sound bank in SoundFont 2 or Downloadable Sounds format that is of decent quality is about 4 million bytes in size.

<sup>98</sup><https://github.com/KristofferKarlAxelEkstrand/AKWF-FREE>

<sup>99</sup><https://github.com/apple/openjdk/blob/xcodejdk14-release/src/java.desktop/share/classes/com/sun/media/sound/EmergencySoundbank.java>

<sup>100</sup><https://peteroupc.github.io/classic-wallpaper>

<sup>101</sup>[https://peteroupc.github.io/classic-wallpaper/docs/uielements.html#Button\\_and\\_Border\\_Drawing\\_Challenge](https://peteroupc.github.io/classic-wallpaper/docs/uielements.html#Button_and_Border_Drawing_Challenge)

## 4.4 Guide for creating 3-D models in the pre-2000 style

Develop a guide for creating 3-D models for use in modern video games that follow the **specification** given earlier on classic (pre-2000) 3-D graphics, in a similar vein to “**Game-Ready 3D Models: Requirements, Creation, and Export**”<sup>102</sup> <sup>103</sup>. Notably, no shader-based techniques should be required for any such models, and advice should apply to models for a game just as though the game were developed in 1999 (or an earlier year) rather than today, but the use of modern creation tools is allowed. (For example, instead of normal, roughness, or ambient-occlusion maps, late-1990s 3-D game models typically employed light maps and bump maps, and such models were generally much coarser than today’s models.)

## 4.5 Book-form tutorial on pre-2000 computer graphics programming

Write a free and open-source tutorial on game graphics programming using the pre-2000 graphic style emphasized in the **specification**<sup>104</sup> given earlier, with the following features:

1. The tutorial is book-length, printable, and programming-language neutral.
2. The tutorial should cover all 2-D and 3-D concepts needed to exploit the specification fully, such as the concepts of tiles, sprites, 2-D layers, textures, 3-D triangles, and features for 3-D graphics. Coverage of graphics programming with a limited-color frame buffer (such as 16 colors or 256 colors) is recommended.
3. The tutorial explains math concepts to readers as necessary, without assuming prior knowledge of math higher than basic algebra.
4. The tutorial does not require the use of hardware acceleration, OpenGL, or Direct3D in the practice exercises. (See definition of “rendered in software” in the specification.)
5. The tutorial should include checks for understanding, guided practice, and independent practice exercises. The tutorial may also include review (retrieval practice) of past concepts.
6. As a strong recommendation, the guided and independent practice can lead to readers writing a feature-rich software renderer that implements the pre-2000 graphics specification without third-party software libraries.

Similar tutorials are *Computer Graphics from Scratch*<sup>105</sup> by Gabriel Gambetta and the **tiny-renderer tutorial**<sup>106</sup> by Dmitry Sokolov, although they are not always geared toward the visuals in pre-2000 video games. For example, Gambetta covers ray tracing (which is outside this specification’s scope) but not sprites or other 2-D concepts.

# 5 Acknowledgments

I acknowledge the advice of the `gameengine devs` community on Reddit.

# 6 License

Any copyright to this page is released to the Public Domain. In case this is not possible, this page is also licensed under **Creative Commons Zero**<sup>107</sup>.

# 7 Notes

---

<sup>102</sup><https://threedium.io/3d-model/game-ready>

<sup>103</sup>The “Game-Ready 3D Models” guide was designed for high-system-resource games from 2024 or so, but appears to have been generated by artificial-intelligence tools, which are not allowed for this project.

<sup>104</sup>[https://peteroupc.github.io/graphics.html#Graphics\\_Challenge\\_for\\_Classic\\_Style\\_Games](https://peteroupc.github.io/graphics.html#Graphics_Challenge_for_Classic_Style_Games)

<sup>105</sup><https://gabrielgambetta.com/computer-graphics-from-scratch/>

<sup>106</sup><https://haqr.eu/tinyrenderer/>

<sup>107</sup><https://creativecommons.org/publicdomain/zero/1.0/>